Keywords: explorative modeling • generative pretraining • mode commitment • sample efficiency

Explorative Modeling: Unlocking a Third Pretraining Axis and End-to-End Generation

Training Generative Models to Commit to Modes by Sampling K Candidates and Updating Only the Best—Yielding 6× Sample Efficiency at Near-SOTA Quality

By Alexi Gladstone, Heng Ji, Yilun Du (University of Illinois Urbana-Champaign; MIT)

arXiv:2607.27372 • July 29, 2026

Generative model training has two well-known scaling axes: parameters and data. A new paper proposes a third—exploration—by sampling K candidate outputs per training example and updating only on the best match. **Explorative Models (XMs)** reach 6.2× sample efficiency, 4.1× FLOP efficiency, and 47% better parameter efficiency compared to standard training, with a near-SOTA FID of 1.43 on ImageNet 256×256 without guidance.

AT A GLANCE

Problem: Standard MLE training averages over modes of the data distribution, producing blurry or incoherent generations.

Why it matters: Mode averaging is a fundamental inefficiency in generative training—fixing it unlocks major sample and FLOP savings.

Method: Sample K candidates per training example; update only on the best-matching one, so gradients reinforce mode-committing outputs.

Result: 6.2× sample efficiency, 4.1× FLOP efficiency, 47% parameter savings, FID 1.43 on ImageNet.

Evidence: Gains are monotone in K across images, video, and language; advantage holds in large-scale pretraining.

The Big Picture

The deep learning revolution was built on end-to-end training: instead of decomposing a problem into hand-designed stages, a single model is trained end-to-end by gradient descent on a task loss. Generative modeling has remained a partial exception. While diffusion models, autoregressive models, and flow models have made remarkable progress, the training loop itself has remained standard: minimize the expected loss over all data examples, averaging over all the modes in the distribution.

This paper argues that the averaging implicit in maximum likelihood estimation is a fundamental inefficiency. When a training example has multiple plausible completions, the gradient pushes the model to cover all of them simultaneously—producing predictions that blur across modes rather than committing to one. The proposed fix, Explorative Modeling (XM), factors the training loop to commit to modes: sample K candidates, pick the best, update on that. Exploration is treated as a quantitative scaling axis alongside parameters and data.

Why Existing Approaches Fall Short

Maximum likelihood training minimizes

$$\mathcal{L}_{\text{MLE}}(\theta) = -\mathbb{E}_{(x,c)\sim\mathcal{D}}\big[\log p_\theta(x \mid c)\big]$$

This encourages the model to assign probability mass to every mode of the data distribution. In practice, this causes mode averaging: the model hedges rather than commits, producing blurry images, inconsistent video frames, or generic text.

Existing remedies address the symptom at inference time (guidance, best-of-N sampling) but leave the training objective unchanged. This means mode-averaging gradients are incorporated into model weights, requiring extra inference compute to overcome them. XM addresses the root cause at training time.

Core Mechanism

At each training step, instead of computing a loss from one generation matched to one data example, XM:

1. Samples K candidate generations $\hat{x}_1, \ldots, \hat{x}_K \sim p_\theta(\cdot \mid c)$.
2. Selects the best-matching candidate: $\hat{x}^* = \arg\max_k \text{sim}(\hat{x}_k, x)$.
3. Computes the training gradient only on $\hat{x}^*$.

The key invariant: gradients always reinforce mode-committing outputs. The model is never penalized for failing to cover modes it did not select. For $K = 1$, XM reduces to standard MLE; increasing K monotonically improves training quality.

Technical Formulation

Let $p_\theta(x \mid c)$ be the generative model. The XM objective is:

$$\mathcal{L}_{\text{XM}}(\theta) = -\mathbb{E}_{(x,c)\sim\mathcal{D}}\big[\log p_\theta(\hat{x}^* \mid c)\big]$$

where

$$\hat{x}^* = \arg \max_{\hat{x}_k \in \{\hat{x}_1,\ldots,\hat{x}_K\}} \text{sim}(\hat{x}_k,\, x).$$

The similarity function $\text{sim}(\cdot,\cdot)$ is task-dependent: perceptual distance for images, video quality metrics for video, and ROUGE/token-overlap for language. The gradient is:

$$\nabla_\theta \mathcal{L}_{\text{XM}} = -\nabla_\theta \log p_\theta(\hat{x}^* \mid c)$$

—identical in form to MLE but computed only on the best candidate.

The authors show that $\mathcal{L}_{\text{XM}}$ upper-bounds the negative log-likelihood of the best-of-K sample. As $K \to \infty$, the objective converges to the entropy of the highest-probability mode, incentivizing mode commitment rather than mode coverage.

Learning or Inference Procedure

Training is identical to standard generative model training except for the sampling step. No changes are needed to the architecture, optimizer, or data pipeline. The per-step compute cost increases by a factor of approximately K for forward passes (to generate K candidates), but only one backward pass is performed, so the memory overhead is modest.

At inference, the model is used identically to a standard generative model: single forward pass, no additional ranking or sampling. Exploration is baked into the parameters at training time, not deferred to test time.

What the Guarantee Says

For $K = 1$, $\mathcal{L}_{\text{XM}} = \mathcal{L}_{\text{MLE}}$. For $K > 1$, $\mathcal{L}_{\text{XM}} \leq \mathcal{L}_{\text{MLE}}$ —the XM objective is never worse, and strictly better when the model has non-zero variance over candidates. The gap grows with the diversity of the model's distribution: early in training, when the model is uncertain, exploration provides the largest benefit.

Experimental Findings

ImageNet 256×256 generation (FID, lower is better):

Model	FID	Guidance
Standard MLE baseline	2.21	No
XM ($K = 8$)	**1.43**	No
XM ($K = 8$) + CFG	**1.41**	Yes
SOTA diffusion model	1.40	Yes

Efficiency at matched FID (FID = 2.21):

Axis	XM vs. Standard
Sample efficiency	6.2×
FLOP efficiency	4.1×
Parameter efficiency	47% fewer params

Scaling of exploration (K): Increasing $K \in \{1, 2, 4, 8, 16, 32\}$ monotonically reduces FID, with approximately log-linear improvement.

Video generation: XM reduces FVD by 21% relative to the MLE baseline at matched compute on a text-to-video benchmark.

Language generation: XM improves the diversity-quality Pareto frontier by 15% MAUVE on a causal LM fine-tuning task.

Ablations and Interpretation

Similarity metric: Replacing perceptual loss with L2 pixel distance degrades FID by 0.6, confirming that the quality of the match function determines how well candidates proxy for mode quality.

K vs. additional training steps: At a fixed total FLOP budget, $K = 4$ with more steps outperforms $K = 1$ with fewer steps, confirming that exploration is more efficient than additional data processing when the model's own distribution spans the relevant modes.

Connection to best-of-N: Best-of-N at inference requires N forward passes per generation and does not affect model weights. XM amortizes the exploration cost into training: the inference model inherits mode-committing behavior without any per-generation overhead.

End-to-end generation: XM stabilizes joint training of encoder-decoder architectures with shared parameters across intermediate and final outputs, because mode-committing gradients prevent the latent space from collapsing toward averaged representations.

Reference

Alexi Gladstone, Heng Ji, Yilun Du. **Explorative Modeling: Unlocking a Third Pretraining Axis and End-to-End Generation**. arXiv:2607.27372, July 29, 2026. https://arxiv.org/abs/2607.27372

Keywords: vision-language-action • robot policy • real-time inference • efficient VLA

TurboVLA: Real-Time Vision-Language-Action Model at 32 Hz on an RTX 4090 with <1 GB VRAM

Eliminating the LLM Bottleneck in Robot Policies via Direct V+L→A Mapping Yields 97.7% on LIBERO with Only 0.2B Parameters

By Hengyi Xie, Chenfei Yao, Xianjin Wu, Xuanyang Xi, Yiping Tang, Di Xu, Yingying Zhu, Dingkang Liang, Xiang Bai, Han Ding (Huazhong Univ. of Science and Technology; Huawei Technologies)

arXiv:2607.27205 • July 30, 2026

State-of-the-art robot manipulation models route all perception through a billion-parameter language model—incurring hundreds of milliseconds of latency and gigabytes of VRAM at every policy step. **TurboVLA** breaks from this convention, replacing the V→L→A pathway with a direct V+L→A mapping using a compact bidirectional interaction module. The result runs at 32 Hz on a consumer RTX 4090 with <1 GB VRAM while achieving 97.7% average success on LIBERO—matching models 35× larger.

AT A GLANCE

Problem: LLM-centric VLA policies incur prohibitive compute and memory overhead at every inference step, blocking real-time deployment.

Why it matters: Real-time control at 32 Hz on consumer hardware is a prerequisite for affordable robot deployment outside research labs.

Method: Independent visual and language encoders fused via lightweight bidirectional cross-attention; continuous action chunks predicted by a compact decoder.

Result: 97.7% on LIBERO, 31.2 ms per step, 0.9 GB VRAM, 0.2B parameters.

Evidence: Matches or exceeds OpenVLA-7B and Pi-0 (3.3B) while running 7–35× faster and consuming 7–16× less memory.

The Big Picture

Vision-language-action models represent the current frontier of generalist robot policies: by building on pre-trained vision-language models, they inherit rich semantic representations that generalize across diverse manipulation tasks. Papers like OpenVLA and Pi-0 have demonstrated that leveraging multi-billion-parameter LLM backbones as the core of a robot policy can yield impressive task success rates.

The bottleneck, however, is clear: inference in these models runs at 2–10 Hz on research-grade GPU hardware, requiring tens of gigabytes of VRAM. Consumer robots, surgical systems, and industrial manipulators demand 30–100 Hz control with modest compute budgets. TurboVLA challenges the architectural assumption that the LLM must be in the critical path of every policy inference.

Why Existing Approaches Fall Short

The dominant V→L→A paradigm processes visual observations by projecting them into the LLM's token space: visual patches become "visual tokens" that attend to language tokens in the LLM's self-attention layers. This design has three costs:

Quadratic attention complexity. The joint visual-language sequence has $N_v + N_l$ tokens; self-attention costs $O((N_v + N_l)^2)$ per layer. For $N_v = 256$ visual tokens and $N_l = 30$ language tokens, this is $\sim 80{,}000$ token pairs per layer, repeated across all 32+ LLM layers.

Memory dominated by LLM weights. Even a 7B-parameter LLM at 4-bit quantization occupies ~ 3.5 GB—more than most embedded compute platforms support.

No separation of concerns. The LLM processes both semantic instruction understanding and low-level observation encoding in the same computational graph, making it impossible to specialize or prune either component independently.

Core Mechanism

TurboVLA introduces three specialized modules that replace the monolithic LLM backbone:

1. **Visual encoder (ViT-S/16):** Encodes the observation image into $N_v = 196$ patch tokens $V \in \mathbb{R}^{N_v \times d_v}$.
2. **Language encoder (CLIP text, frozen):** Encodes the instruction into $N_l \approx 30$ tokens $L \in \mathbb{R}^{N_l \times d_l}$.
3. **Bidirectional Vision-Language Interaction (BiVLI):** Applies two rounds of alternating cross-attention, letting V attend to L and L attend to V, producing aligned representations V' and L'.

The BiVLI module contains only ∼50M parameters. The action decoder takes $[\text{pool}(V') \oplus \text{pool}(L')]$ and predicts a chunk of $H = 10$ continuous actions.

Technical Formulation

For round $r \in \{1, 2\}$ of BiVLI:

$$V^{(r)} = V^{(r-1)} + \text{MHA}_{V \leftarrow L}(Q{=}V^{(r-1)},\ K{=}L^{(r-1)},\ V{=}L^{(r-1)})$$

$$L^{(r)} = L^{(r-1)} + \text{MHA}_{L \leftarrow V}(Q{=}L^{(r-1)},\ K{=}V^{(r)},\ V{=}V^{(r)})$$

where MHA is multi-head attention. The interaction cost is $O(N_v \cdot N_l)$ per round—30× lower than full self-attention over the combined sequence.

The combined representation $z = \text{pool}(V^{(2)}) \oplus \text{pool}(L^{(2)})$ is decoded to an action chunk:

$$\hat{a}_{1:H} = \text{Decoder}(z), \qquad \hat{a}_{1:H} \in \mathbb{R}^{H \times d_a}$$

Training minimizes the behavior cloning loss:

$$\mathcal{L}_{\text{BC}} = \sum_{h=1}^{H} \left\| \hat{a}_h - a_h^* \right\|_2^2$$

over expert demonstrations.

Learning or Inference Procedure

Training uses two stages. In the first (pretraining) stage, the visual encoder is initialized from CLIP ViT-S/16 and the BiVLI module is randomly initialized; both are trained on the OpenX-Embodiment dataset (∼1M demonstrations). In the second (fine-tuning) stage, the full model including the visual encoder is fine-tuned on task-specific demonstrations in the target environment (LIBERO, ∼500 demonstrations per suite). The CLIP text encoder is frozen throughout.

At inference, the visual encoder, BiVLI, and action decoder run in a single forward pass per time step—no autoregressive decoding, no KV cache management, no LLM overhead.

What the Guarantee Says

By design, the backbone parameters p_θ are never called at inference time. The only per-step compute is ViT-S forward (12 ms), BiVLI two-round cross-attention (11 ms), and MLP action decoder (8 ms), totaling 31.2 ms—safely below the 33 ms deadline for 30 Hz control. This is a structural bound, not a statistical approximation: no matter how complex the manipulation task, the per-step latency is fixed by architecture rather than by task difficulty.

Experimental Findings

LIBERO benchmark (5 suites, higher is better):

Model	Params	Avg.	ms/step	VRAM
OpenVLA-7B	7B	96.1%	218	14.0 GB
Pi-0 (distil.)	3.3B	97.2%	82	6.4 GB
TurboVLA	**0.2B**	**97.7%**	**31**	**0.9 GB**

TurboVLA achieves the best average success rate while using 35× fewer parameters, running 7× faster, and consuming 7–16× less memory.

Cross-embodiment: Evaluated on Franka, xArm, and UR5, TurboVLA shows less catastrophic forgetting between embodiments than OpenVLA, attributed to the frozen CLIP text encoder providing a stable semantic anchor during multi-embodiment fine-tuning.

Ablations and Interpretation

BiVLI rounds: $R = 1$ round drops average success by 3.2%; $R = 4$ rounds adds only 0.4% while doubling BiVLI latency. Two rounds is the Pareto-optimal choice for accuracy/latency trade-off.

Visual encoder size: ViT-B/16 (3× more parameters) improves success by 1.1% but doubles VRAM. ViT-S/16 is appropriate for resource-constrained deployment.

Action chunk horizon: $H = 10$ gives the best task completion; shorter horizons ($H = 4$) hurt due to high-frequency control jitter and longer horizons ($H = 20$) hurt due to compounding prediction error.

Reference

Hengyi Xie, Chenfei Yao, Xianjin Wu, Xuanyang Xi, Yiping Tang, Di Xu, Yingying Zhu, Dingkang Liang, Xiang Bai, Han Ding. **TurboVLA: Real-Time Vision-Language-Action Model at 32 Hz on an RTX 4090 with <1 GB VRAM**. arXiv:2607.27205, July 30, 2026. https://arxiv.org/abs/2607.27205

Keywords: code generation • reinforcement learning • runtime efficiency • execution feedback

RLPF: Reinforcement Learning from Performance Feedback for Code Generation

A Staged Reward Trains Code Models to Prefer Faster Correct Programs—Raising Correct-and-Efficient Rate from 8.1% to 38.6%

By Huihao Jing, Haozhe Cui, Wenbin Hu, Shaojin Chen, Haochen Shi, Changxuan Fan, Yuxuan Liu, Hanyu Yang, Sirui Zhang, Ziyi Chen, Haoran Li, Yangqiu Song (Hong Kong Univ. of Science and Technology)

arXiv:2607.27271 • July 29, 2026

Code models trained with execution feedback reward correctness—not efficiency. Two programs can pass the same test suite while differing by orders of magnitude in runtime. **RLPF** (Reinforcement Learning from Performance Feedback) introduces a staged reward: correctness is gated first; passing programs then receive a speedup signal relative to a reference solution. On a competitive programming benchmark, RLPF raises a base model from 8.1% to 38.6% CGRE (Correct-and-Graded Relative Efficiency), closing two-thirds of the gap to a frontier model.

AT A GLANCE

Problem: Correctness-only RL for code leaves no signal for runtime efficiency; programs that pass tests may be asymptotically slow.

Why it matters: Systems code, competitive programming, and latency-sensitive applications require efficient implementations as a primary objective.

Method: Two-stage reward: correctness gate first; relative speedup over a reference solution for programs that pass.

Result: Correct-and-runnable rate 11.1%→54.6%; CGRE 8.1%→38.6%.

Evidence: Staged reward outperforms correctness-only RL by 7.3 CGRE points; validated across array, sorting, graph, and DP tasks.

The Big Picture

The code generation field has converged on a compelling training paradigm: train a language model on execution feedback. Sample programs from the model, execute them against a test suite, and use the pass/fail signal as a reward for reinforcement learning. This has driven dramatic improvements on correctness-centric benchmarks such as HumanEval, MBPP, and LiveCodeBench.

But correctness is not the only property that matters. In systems programming, competitive programming, and any latency-sensitive application, a correct but slow program is often not good enough. Runtime efficiency should be a first-class training objective. RLPF is the first study that makes efficiency a primary RL training signal for code generation, rather than an evaluation-time metric.

Why Existing Approaches Fall Short

Correctness-only RL has a critical gap: the model learns to produce any program that passes the test suite, regardless of runtime. For most benchmarks, the test suite uses small inputs where even $O(n^3)$ solutions pass within the time limit. The model thus learns to write correct but asymptotically slow code.

Three properties of runtime make it a difficult RL reward:

1. Fragility. Runtime is only meaningful after a program compiles and passes all correctness checks. If 80% of sampled programs fail to compile, a pure efficiency reward provides near-zero gradient.

2. Non-comparability across tasks. An absolute runtime of 100 ms may be excellent for one problem and catastrophic for another. A relative metric is required.

3. Mode collapse risk. Optimizing for efficiency without a correctness constraint causes the model to produce fast-but-wrong programs (e.g., always returning a hard-coded answer).

RLPF addresses all three with a staged reward design.

Core Mechanism

RLPF uses PPO with a two-stage reward function:

Stage 1 — Correctness gate. A sampled program $\hat{c}$ is executed against the full test suite. If it fails to compile, fails any test, or times out, it receives zero reward and the performance stage is skipped.

Stage 2 — Performance reward. If $\hat{c}$ passes all tests, its runtime $t(\hat{c})$ is measured on a distribution of inputs. The reward is the relative speedup over a reference correct solution:

$$r_{\text{perf}}(\hat{c}) = \frac{t_{\text{ref}} - t(\hat{c})}{t_{\text{ref}}}$$

This is bounded in $(-\infty, 1]$, with 0 meaning same speed as the reference and 1 meaning infinite speedup.

Technical Formulation

The combined reward is:

$$r(\hat{c}) = \alpha \cdot r_{\text{correct}} + (1-\alpha) \cdot r_{\text{perf}}(\hat{c}) \cdot \mathbf{1}\big[r_{\text{correct}} = 1\big]$$

where α is annealed from 1.0 to 0.3 over training via a cosine schedule.

The PPO objective maximizes:

$$\mathcal{J}(\theta) = \mathbb{E}[\min(\rho_t A_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t)] - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$$

where $\rho_t = \pi_\theta(\hat{c}|p)/\pi_{\text{old}}(\hat{c}|p)$ and A_t is the advantage from $r(\hat{c})$.

The evaluation metric CGRE is:

$$\text{CGRE} = \frac{1}{|P|} \sum_{p \in P} \mathbf{1}[\hat{c}_p \text{ passes}] \cdot \max\left(0,\ 1 - \frac{t(\hat{c}_p)}{t_{\text{ref}}(p)}\right)$$

This rewards both correctness and efficiency: failing programs contribute 0, and faster programs contribute proportionally more.

Learning or Inference Procedure

Training uses a dataset of competitive programming problems covering arrays, sorting, graph algorithms, and dynamic programming. Reference solutions are taken from accepted competitive submissions.

PPO is run for 3,000 steps with batch size 32, sampling 8 rollouts per problem per step. Each rollout is compiled and executed; the staged reward is computed online. The α annealing follows a cosine curve, ensuring the correctness stage dominates early training when the model produces mostly incorrect programs.

The KL divergence penalty $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ relative to the initial model prevents mode collapse onto a few fast-but-narrow templates.

What the Guarantee Says

The α annealing provides a correct-first guarantee: during the first phase of training (α near 1), the policy is incentivized exclusively to produce correct programs. By the time α decreases and the performance reward becomes significant, the correctness rate is already high enough that most sampled programs enter the performance stage, providing dense efficiency gradients.

Experimental Findings

Method	Correct-and-Run	CGRE
Base model	11.1%	8.1%
Correctness-only RL	47.3%	18.4%
RLPF (staged)	**54.6%**	**38.6%**
GPT-5.4 (zero-shot)	58.2%	41.1%

RLPF raises CGRE by 30.5 points over the base model and by 20.2 points over correctness-only RL. The gap to a frontier closed-source model is reduced to 2.5 CGRE points.

By problem category: Largest CGRE gains occur on array and DP problems (44.2% CGRE), where efficient algorithmic patterns (prefix sums, memoization) are learnable. Smaller gains on graph problems (28.1%), where efficiency requires sophisticated data structures harder to induce from supervision.

Ablations and Interpretation

Reward annealing: Fixing $\alpha = 0.5$ throughout reduces CGRE by 8.2 points, as the efficiency signal disrupts early correctness learning and produces more incorrect programs.

Reference quality: Using a 10th-percentile runtime reference (suboptimal) reduces CGRE by 4.1 points—reference quality is a primary bottleneck and should be set to the best known solution.

KL penalty: Removing D_{KL} causes collapse onto 3–5 high-efficiency code templates, reducing CGRE diversity and failing on problems that do not fit those templates. Performance drops by 6.3 CGRE.

Number of rollouts: 8 rollouts per problem per step is the sweet spot; 4 rollouts degrades CGRE by 3.1 points due to lower variance in the reward signal.

Reference

Huihao Jing, Haozhe Cui, Wenbin Hu, Shaojin Chen, Haochen Shi, Changxuan Fan, Yuxuan Liu, Hanyu Yang, Sirui Zhang, Ziyi Chen, Haoran Li, Yangqiu Song. **RLPF: Reinforcement Learning from Performance Feedback for Code Generation**. arXiv:2607.27271, July 29, 2026. https://arxiv.org/abs/2607.27271

Keywords: alignment tax • domain adaptation • parametric memory • catastrophic forgetting

MemSFT: Mitigating Alignment Tax with an External Parametric Memory

A Plug-and-Play Memory Module Imparts Domain Knowledge Without Touching Backbone Weights—Matching Full SFT While Preserving General Capability

By Jiarui Wang, Xiang Shi, Jiaqi Cao, Rubin Wei, Xiquan Wang, Hao Sun, Jingzhi Wang, Zhiqi Yang, Qipeng Guo, Bowen Zhou, Zhouhan Lin (Shanghai Jiao Tong University; Fudan University; Tencent AI Lab)

arXiv:2607.25614 • July 28, 2026

Fine-tuning an LLM on a specialized domain reliably degrades its general-purpose capability—a phenomenon called the alignment tax. **MemSFT** avoids this by never updating backbone parameters: a compact parametric memory module is trained to imitate a domain retriever, and a learned router fuses memory and backbone distributions token-by-token at inference. Across biology, geoscience, and law, MemSFT exceeds full SFT on domain tasks while degrading general benchmarks by only 0.2%— versus a 20.5-point drop for full SFT.

AT A GLANCE

Problem: Fine-tuning on domain data causes catastrophic forgetting of general capabilities, limiting the practical utility of LLM adaptation.

Why it matters: Deploying LLMs in law, medicine, and science requires domain expertise without sacrificing general reasoning.

Method: Train a small parametric memory to distill a domain retriever; a router dynamically fuses memory and backbone at each decoding step.

Result: 72.6 domain avg. (vs. 71.8 for full SFT); 74.6 general avg. (vs. 54.3 for full SFT).

Evidence: Evaluated on biology, geoscience, law with Qwen3-8B through Qwen3-235B-A22B; memory transfers across model sizes.

The Big Picture

Large language models have transformed the landscape of domain-specific AI: a single pretrained model can serve as the foundation for law, medicine, finance, and science—but only with significant customization. Full supervised fine-tuning (SFT) on domain data is the dominant adaptation method, but it carries a well-documented cost: catastrophic forgetting. General benchmarks drop 15–30% relative to the base model after full SFT on a specialized corpus.

MemSFT recasts domain adaptation as a knowledge injection problem. Instead of modifying the backbone, it adds a compact module that holds the domain knowledge. The backbone remains intact; the memory supplies the domain delta; a router blends them conditionally during generation.

Why Existing Approaches Fall Short

Three adaptation paradigms exist, each with a critical limitation:

Full SFT: Updates all backbone parameters. Achieves strong domain performance but causes catastrophic forgetting (general benchmarks drop 15–30%). Once fine-tuned, the backbone cannot easily serve multiple domains simultaneously.

LoRA/adapters: Updates low-rank adapter matrices ($\ll 1\%$ of parameters). Reduces forgetting but does not eliminate it; domain gains are weaker than full SFT.

Retrieval-augmented generation (RAG): Retrieves relevant documents at inference time. Preserves general capability but adds retrieval latency ($\sim$100–500 ms per query), requires a maintained document store, and does not internalize patterns that allow efficient pattern completion.

MemSFT achieves full SFT domain performance with RAG-level general capability preservation, and with only 15% additional inference latency over a vanilla backbone—no retrieval system required.

Core Mechanism

MemSFT consists of two components trained sequentially without backbone modification:

Parametric memory M_ϕ: A compact transformer ($\sim$75M parameters) trained to imitate a dense retriever operating over domain documents. Given the generation prefix $h_{1:t}$, M_ϕ produces a distribution over the next token that reflects domain-relevant patterns.

Router g_ψ: A lightweight gating network ($\sim$1M parameters) that predicts, at each decoding step, what fraction of the output distribution should come from the memory vs. the backbone.

Technical Formulation

The memory is trained by knowledge distillation from a retriever:

$$\mathcal{L}_{\text{mem}} = D_{\text{KL}}\big(\text{Retriever}(\cdot \mid h_{1:t}, \mathcal{D}_{\text{dom}}) \,\|\, M_\phi(\cdot \mid h_{1:t})\big)$$

where $\mathcal{D}_{\text{dom}}$ is the domain corpus. The retriever is a dense retrieval model that augments a frozen LM with top-k documents.

The final token distribution is:

$$p(w_t \mid w_{<t}) = g_\psi(w_{<t}) \cdot M_\phi(w_t \mid w_{<t}) + \big(1 - g_\psi(w_{<t})\big) \cdot p_\theta(w_t \mid w_{<t})$$

where p_θ is the frozen backbone and $g_\psi(w_{<t}) \in [0, 1]$.

The router is trained to maximize the likelihood of domain data under $p(\cdot)$, with a sparsity regularizer:

$$\mathcal{L}_{\text{router}} = -\mathbb{E}_{(w,c)\sim\mathcal{D}_{\text{dom}}}\big[\log p(w \mid c)\big] + \lambda \|g_\psi\|_1$$

Learning or Inference Procedure

Training proceeds in two stages:

Stage 1 (memory distillation): Train M_ϕ against the retriever output distribution on domain documents for 10K steps with Adam. Backbone is frozen throughout.

Stage 2 (router training): Freeze M_ϕ, train g_ψ on a held-out domain set for 1K steps. The sparsity regularizer encourages the router to invoke memory selectively: on non-domain prompts, the router converges to $g_\psi \approx 0$ (pure backbone).

At inference, the backbone and memory run in parallel; the router blends their outputs. Per-step latency overhead is $\approx 15\%$ for Qwen3-8B.

What the Guarantee Says

Since the backbone parameters θ are never modified, the backbone's general-task performance is preserved exactly by construction. Any degradation in general benchmarks must arise from the router incorrectly invoking domain memory for non-domain prompts. The sparsity regularizer controls this: the expected general degradation is bounded above by $\lambda \cdot \mathbb{E}[g_\psi(w_{<t})]$ on non-domain contexts, which converges to near-zero with appropriate λ.

Experimental Findings

Domains: Biology (PubMedQA, BioASQ), Geoscience (GeoQA), Law (LegalBench). Models: Qwen3-8B to Qwen3-235B-A22B.

Method	Domain avg.	General avg.
Base LLM	62.3	74.8
Full SFT	71.8	54.3
LoRA	68.4	70.1
RAG	70.2	74.7
MemSFT	**72.6**	**74.6**

MemSFT exceeds full SFT on domain tasks by 0.8 points while preserving general capability (only 0.2-point drop versus a 20.5-point drop for full SFT). It also exceeds RAG by 2.4 domain points with 3× lower inference latency.

Cross-model reuse: A memory trained with Qwen3-8B transfers to Qwen3-72B, recovering 92% of per-domain performance vs. a native Qwen3-72B memory—demonstrating that the memory is architecturally portable across backbone sizes.

Ablations and Interpretation

Router sparsity: Removing $\lambda\|g_\psi\|_1$ causes the router to always blend both distributions ($g_\psi \approx 0.5$), degrading general performance by an additional 3.1 points while providing only marginal domain gains.

Memory size: Scaling from 25M to 100M parameters improves domain performance by 2.4 points; diminishing returns beyond 75M.

Distillation teacher quality: BM25 retriever (sparse) as teacher vs. dense retrieval reduces domain performance by 3.8 points, confirming that retrieval quality is the primary bottleneck for memory fidelity.

Reference

Jiarui Wang, Xiang Shi, Jiaqi Cao, Rubin Wei, Xiquan Wang, Hao Sun, Jingzhi Wang, Zhiqi Yang, Qipeng Guo, Bowen Zhou, Zhouhan Lin. **MemSFT: Mitigating Alignment Tax with an External Parametric Memory**. arXiv:2607.25614, July 28, 2026. https://arxiv.org/abs/2607.25614

Keywords: agentic reinforcement learning • skill evolution • cross-task transfer • credit assignment

SkillRise: Agentic Reinforcement Learning for Cross-Task Skill Evolution

A Single Policy Alternates Between Solving Tasks and Curating a Growing Skill Document—Rewarded for Downstream Impact, Not Document Plausibility

By Zhiyuan Yao, Yuxin Chen, Zhengxi Lu, Zishan Xu, Yueqing Sun, Yifu Guo, Yuquan Lu, Zhengzhou Cai, Kangning Zhang, Zhuowen Han, Zihan Wang, Ziang Ye, Qi Gu, Xunliang Cai, Weiwen Liu, Yongliang Shen (Zhejiang University; NUS; SJTU; Meituan)

arXiv:2607.26784 • July 29, 2026

Standard agentic RL treats each task as an independent episode, discarding transferable workflows after every interaction. **SkillRise** trains a single policy to simultaneously solve tasks and maintain an evolving natural-language skill document. A key innovation is *decoupled credit assignment*: skill curation is rewarded based on its downstream impact on future tasks, not on document quality alone. On ALFWorld, WebShop, and ScienceWorld, SkillRise achieves the best Pass@1 among all compared methods.

AT A GLANCE

Problem: Agentic RL discards generalizable patterns after each task, preventing efficient transfer to future related tasks.

Why it matters: Real-world agents face streams of related tasks; the ability to curate and reuse skills is essential for continual improvement.

Method: Single policy that solves tasks and curates a skill document; decoupled credit assignment rewards curation by downstream task improvement.

Result: Best Pass@1 on ALFWorld (78.9%), WebShop (66.3%), ScienceWorld (50.2%).

Evidence: 11–12 pp. gain over standard RLVR; shared solve-curate policy outperforms separate-policy baselines.

The Big Picture

Agentic reinforcement learning for LLMs (RLVR) has demonstrated that language agents can learn to navigate web interfaces, plan household tasks, and execute scientific workflows purely from outcome-based rewards. But these agents are trained and tested on episodic tasks: each episode is independent, and the knowledge gained in one episode is only preserved insofar as it is absorbed into the model's parameters through gradient updates.

This means every new task starts from scratch. A household agent that learned to find items in a drawer on task 1 has no mechanism to record that workflow and apply it directly on task 2. SkillRise proposes a simple but effective alternative: let the agent write its own workflow documentation, and reward the documentation by how much it helps future tasks.

Why Existing Approaches Fall Short

Prior work on skill-augmented agentic RL (Voyager, SKILL1, SkillRL, SkillFlow) decomposes skill evolution into a pipeline:

1. *Extract:* A separate skill extraction module reads completed trajectories and identifies transferable patterns.
2. *Store:* Extracted skills are written to a retrieval database.
3. *Retrieve:* At test time, a retriever fetches relevant skills as context.

This pipeline has two weaknesses. First, it requires separate optimization objectives for each component: extraction, retrieval, and execution are trained separately with potentially conflicting objectives. Second, the skill document is static after extraction and cannot be revised in light of future task failures—a pattern extracted from an easy instance may mislead on a harder variant.

SkillRise collapses all three stages into a single reinforcement learning objective applied to a single policy.

Core Mechanism

SkillRise operates over a sequence of N tasks $T_1, T_2, \ldots, T_N$ organized from easy to hard. For each task T_t, the agent is given the current skill document S_{t-1} and takes two sequential actions:

Solve: Generate a trajectory $a_{1:H}$ to complete T_t given context (T_t, S_{t-1}).

Curate: Given $(T_t, a_{1:H}, S_{t-1})$, generate an updated skill document S_t that adds, revises, or deletes workflow entries in light of what was learned on T_t.

Both actions are produced by the same policy π_θ; no separate extraction or retrieval module exists.

Technical Formulation

Let $r_{\text{solve}}(T_t, a_{1:H})$ be the task completion reward and $r_{\text{curate}}(S_t, T_{t+1:t+k})$ be the discounted impact of the updated skill document on the next k tasks:

$$r_{\text{curate}}(S_t, T_{t+1:t+k}) = \frac{1}{k}\sum_{j=1}^{k} \gamma^j \cdot \Delta r_{\text{solve}}(T_{t+j},\ S_t \text{ vs. } S_{t-1})$$

The SkillRise objective is:

$$\mathcal{J}(\theta) = \mathbb{E}\left[\sum_{t=1}^{N} r_{\text{solve}}(T_t, a_{1:H}^{(t)}) + \gamma \cdot r_{\text{curate}}(S_t, T_{t+1:t+k})\right]$$

This is the critical distinction from prior work: curation is rewarded by downstream impact (Δr_{solve}), not by surface-level quality. A skill is only reinforced when it demonstrably helps solve future tasks.

Learning or Inference Procedure

Training uses Group Relative Policy Optimization (GRPO): multiple curation outputs are sampled for each post-task state, and the policy is updated based on relative advantage between outputs that have better vs. worse downstream impact.

The training curriculum organizes tasks by difficulty within each domain: easier instances first. This ensures that skill documents accumulate useful content early in training before harder tasks demand more nuanced workflows. Skill documents are constrained to 2,000 tokens to fit within the context window alongside task instructions.

What the Guarantee Says

Decoupled credit assignment provides a monotonicity property: if the curation policy is improved, the downstream solve reward is non-decreasing in expectation. This follows from the definition of r_{curate}: if S_t improves performance on future tasks, the curation reward is positive, incentivizing the policy to generate more useful updates.

For a fixed backbone, the SkillRise objective upper-bounds the cumulative task completion rate over the N-task stream.

Experimental Findings

ALFWorld (household task completion, Pass@1):

Method	Pass@1
Base LLM	42.3%
Standard RLVR	67.8%
SKILL1	71.4%
SkillRise	**78.9%**

WebShop (e-commerce navigation, Task Success):

Method	Success
Standard RLVR	56.2%
SkillFlow	61.4%
SkillRise	**66.3%**

ScienceWorld (scientific experiment tasks, Pass@1):

Method	Pass@1
Standard RLVR	38.1%
SKILLC	43.7%
SkillRise	**50.2%**

Gains over standard RLVR are 11.1 pp (ALFWorld), 10.1 pp (WebShop), and 12.1 pp (ScienceWorld).

Ablations and Interpretation

Decoupled vs. immediate curation reward: Replacing r_{curate} with an immediate quality score (document fluency or coverage) reduces ALFWorld Pass@1 by 6.6 pp, confirming that downstream impact is the essential signal.

Shared vs. separate policy: Training a separate curation policy (independent weights) reduces ALFWorld Pass@1 by 4.1 pp, suggesting productive weight-sharing: solving improves curation, and curation improves solving through shared world model knowledge.

Curriculum ordering: Random task ordering reduces ALFWorld Pass@1 by 6.3 pp, confirming that the easy-to-hard curriculum is essential for early skill accumulation before difficult tasks arrive.

Document length: Allowing up to 4,000 tokens (vs. 2,000) gains 1.2 pp on ScienceWorld, suggesting the constraint is occasionally binding for complex scientific workflows.

Reference

Zhiyuan Yao, Yuxin Chen, Zhengxi Lu, et al. **SkillRise: Agentic Reinforcement Learning for Cross-Task Skill Evolution**. arXiv:2607.26784, July 29, 2026. https://arxiv.org/abs/2607.26784